

高级语言程序设计

实验报告

南开大学 计算机大类

姓名：董语晗

学号：2512607

班级：2025 级计算机卓越伯苓班

目录

一、作业题目	1
二、开发软件	1
三、课题要求	1
四、主要流程	2
1. 整体流程概述	2
2. 具体代码实现	2
(1) 数据结构搭建	2
(2) 后端算法	3
(3) 前后端通信	3
3. 单元测试	5
五、收获	5

一、作业题目

单细胞数据分析可视化平台开发

二、开发软件

Qt Creator 19.00、Visual Studio 2022

三、课题要求

1、面向对象

本项目基于 C++ 面向对象思想设计, 以 AnnDataContainer 为核心数据结构, 封装单细胞数据与操作; LassoView、Downsample 等算法独立成类, 实现高内聚、低耦合与模块化复用; 采用前后端分离架构, 利用 Qt、Crow 等实现前后端 API 通信与。

2、单元测试

对核心模块编写单元测试:

AnnDataContainer: 测试构造、拷贝、观测列增删、嵌入获取;

LassoView: 测试细胞扩选算法正确性、seed/propagated 标记;

Downsample: 测试降采样结果尺寸、索引映射有效性;

接口层: 测试 HTTP 接口参数解析、任务创建与状态返回。

3、模型部分

Lasso-View: 基于细胞连通图与标签传播, 实现目标细胞群扩选;

Downsample: 均衡降采样, 保留聚类结构并减少计算量。

4、验证

功能验证: 能够正确上传细胞数据文件, 进行圈选标记细胞功能, 可以切换分类上色标准。Downsample 正常运行, 显示降采样后更加稀疏的 UMAP 图。LassoView 正确生成 lasso_view_status 三色标记; 前端正常加载、绘图、切换颜色、运行分析任务;

结果验证: 扩选细胞数量合理、聚类与可视化一致、任务可下载结果;

稳定性验证: 接口无崩溃、数据拷贝安全、异步任务状态正常更新。

四、主要流程

1、整体流程概述

本项目主要分为三个部分：前端 react 交互界面、后端算法核心实现、以及前后端通信接口。

本项目采用前后端分离架构, 其中前端 react 交互界面由 main.jsx、styles.css、index.html 三个文件构成, 主要由张祯颀学长提供框架。此次大作业主要负责实现后端算法 C++ 版本的开发与改进以及利用 QtHttpServer、Crow、JSON 等库实现前后端 API 通信。

2、具体代码实现

(1) 数据结构搭建

数据结构搭建部分涉及较多用户开发的工具包的使用以及生物信息学文件结构知识, 借助 AI 工具 (DeepSeek、chatGPT、豆包) 进行了思路设计、工具包的获取与应用以及代码实现和修改过程

json_utils.h:

JSON 文件读写辅助工具, 在本项目中主要用于保存 Downsample 下采样后的细胞索引映射文件、读写分析任务的临时结果与配置信息、为后端算法提供本地数据持久化支持。

DatasetStore:

用于存储、管理所有已加载的数据集及其元数据, 便于快速查询而不必加载全部数据。

AnnData:

此文件包含核心的 AnnDataContainer 类, 在 C++ 中统一封装单细胞数据、观测列、降维嵌入、稀疏矩阵等, 并提供标准化数据接口, 封装常用操作, 如获取细胞标签列、获取可用嵌入、获取默认绘图颜色列等, 让上层代码可直接调用数据。

H5adReader:

由于输入的数据是 h5ad 格式, 所以采用 H5adReader 利用 HighFive 包来解析生物信息学中标准的 .h5ad 格式文件, 将二进制单细胞数据, 包括细胞、基因、表达矩阵、聚类结果、UMAP/PCA 降维坐标等, 转换为 C++ 后端内存中可

直接使用的 AnnDataContainer 数据结构并交由 DatasetStore 进行管理。

(2) 后端算法

此平台主要包含两个下采样 (Downsample) 和 LassoView 两个分析模块, Downsample 和 LassoView 都有可参考的开源代码能在 github 上找到。

Downsample:

是一个基于 C++ 实现的数据集下采样工具, 核心目标是对单细胞数据分析中常用的 AnnData 格式数据(主要基于 UMAP 坐标)进行高效、均衡的下采样, 即抽样, 同时保留数据的聚类特征, 并支持从下采样数据的选择结果还原到原始数据集。

LassoView:

基于用户手动选择的少量细胞(种子细胞), 利用细胞间的连接关系(如单细胞测序数据中的细胞相似性网络), 通过标签传播算法, 扩展出与种子细胞特征相似的所有细胞。

(3) 前后端通信

前后端通信是本项目的核心支撑模块, 基于 HTTP 协议与 RESTful API 架构, 使用 JSON 作为数据交换格式, 以 Crow 作为后端框架用于创建 API、接收请求、返回 JSON。前端通过 API 向后端发送数据请求与分析指令, 后端完成数据读取、算法计算与任务管理后, 将结果返回前端进行可视化展示。该模块实现了界面交互与数据分析的解耦, 保证了系统的高效性、稳定性与可扩展性, 是整个单细胞数据分析平台能够正常运行的关键, 主要在 http_server 文件中实现。

本部分在涉及到 Http 协议和具体 Crow 语法、构建 API 的部分使用了 AI 协助。

【1】数据集管理接口

获取数据集列表:

前端请求后, 后端从 DatasetStore 中读取所有已加载数据集, 返回数据集名称、ID、细胞数等信息。

对应路由函数: `CROW_ROUTE(app, "/api/datasets")`

核心依赖类/函数:

DataSetStore 类：数据集存储管理类

DataSetStore::listAll(): 获取所有数据集元信息

QUuid/QString: Qt 字符串与 ID 处理

上传 / 加载数据集：

前端上传 .h5ad 文件，后端通过 H5adReader 解析并构建 AnnDataContainer，存入 DataSetStore 统一管理。

对应路由函数：CROW_ROUTE(app, "/api/upload")

前端获取指定数据集的绘图所需摘要信息，包括细胞数量、可着色列名称、可用嵌入（UMAP/PCA），用于前端初始化绘图、颜色选择下拉框、视图渲染。

获取数据集摘要：

返回数据集的绘图信息、可着色列名、嵌入信息（UMAP/PCA），用于前端绘图展示。

对应路由函数：CROW_ROUTE(app, "/api/datasets/<string>/summary")

核心依赖类 / 函数：

DataSetStore::get(): 根据 ID 获取数据集

AnnDataContainer::getObsColumns(): 获取细胞标签列

AnnDataContainer::getAvailableEmbeddings(): 获取可用嵌入（UMAP/PCA）

AnnDataContainer::defaultEmbeddingKey(): 获取默认绘图嵌入

【2】 分析任务接口（最重要部分）

路由函数：

CROW_ROUTE(app, "/api/datasets/<string>/analysis-jobs").methods(crow::HTTPMethod::Post)

这是最核心的前后端通信接口，功能如下：

- 1) 前端发送：分析类型（LassoView / Downsample）、选中的细胞 ID、算法参数。
- 2) 后端接收后：创建任务（JobStore）进入 running 状态调用算法模块（LassoView::run / Downsample::run）生成新的 AnnData 结果数据集返回任务完成状态与结果。

- 3) 结果返回: 任务状态、结果数据集 ID、新增列名 (如 lasso_view_status)、扩展细胞 ID 列表。

前端收到后自动刷新 View2 绘图。

【3】任务状态轮询接口

前端定时请求: /api/datasets/<string>/analysis-jobs/<string>

后端返回: running (运行中) /completed (已完成) /failed (失败)

任务完成后前端自动加载结果数据。

【4】绘图数据接口

后端从 AnnDataContainer 中提取: UMAP 坐标、颜色标签、细胞索引, 返回给前端用于 Plotly 绘图。

3、单元测试

本项目对核心功能进行了功能测试与逻辑验证, 以确保模块运行正确。

测试内容包括:

(1) 数据结构测试

测试 AnnDataContainer 的构造、数据读取、列获取等功能, 确保数据结构稳定。

(2) 文件解析测试

使用 H5adReader 读取 h5ad 文件, 验证数据解析正确。

(3) 算法功能测试

对 LassoView 细胞扩选、Downsample 下采样与映射还原进行测试, 确保算法输出符合预期。

(4) 前后端接口测试

测试数据集加载、任务提交、状态查询、绘图数据返回等 API 接口, 确保通信正常、数据格式正确。

经测试, 所有核心模块功能正常、逻辑正确、运行稳定, 满足项目设计要求。

五、收获

1、面向对象与多态

通过设计 AnnDataContainer 统一数据结构, 深刻理解封装、继承、多态的意义。统一接口使原始数据集与分析结果数据集可无缝替换, 便于扩展新算法。

2、前后端架构与 API

本次项目实现了单细胞数据分析平台的后端开发与前后端通信交互，让我对前后端分离架构、HTTP 服务开发、API 接口设计有了深刻的实践理解。

首先，我掌握了基于 Crow 框架搭建 C++ 后端 HTTP 服务器的完整流程，理解了 HTTP 协议的请求-响应模型，学会了通过路由分发不同接口请求，实现数据集管理、算法任务提交、结果查询、数据导出等功能，搭建起前端与后端之间的通信桥梁。

其次，通过设计 RESTful 风格的 API 接口，我理解了前后端数据交互的规范。前端通过 JSON 格式传递用户操作参数，后端解析请求、调用数据读取与算法模块完成计算，再将处理结果以 JSON 返回给前端，实现了界面交互与数据分析的解耦。

最后，我体会到模块化与分层设计在通信系统中的重要性，后端只负责业务逻辑与计算，前端专注可视化展示，二者通过标准化接口协同工作，为后续学习打下了坚实基础。